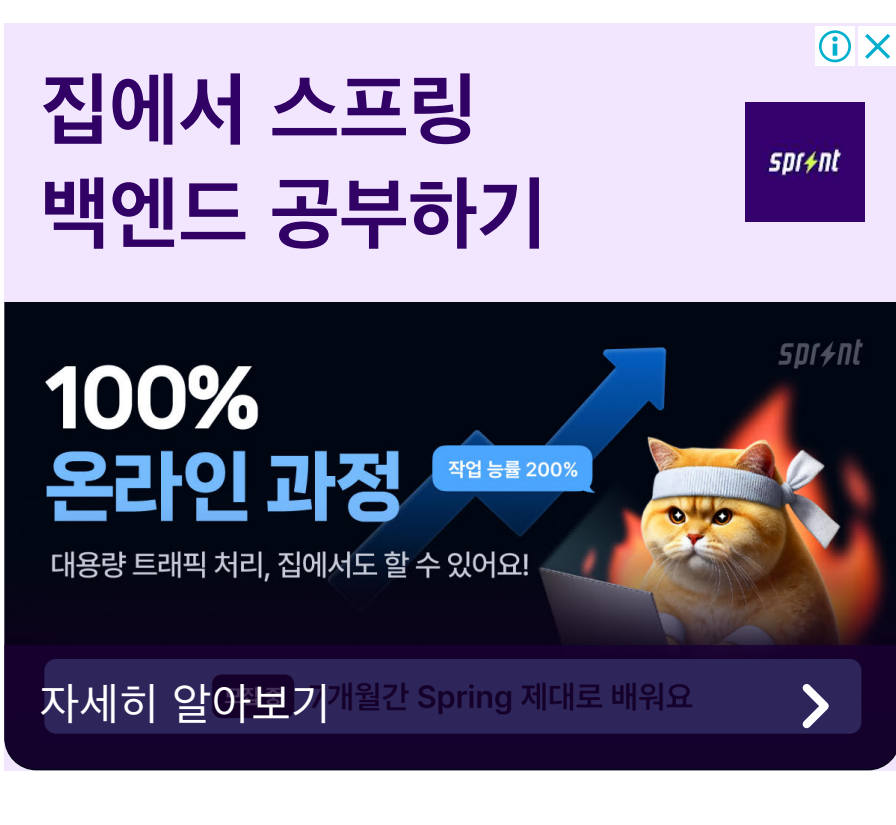


02-07 패딩(Padding)



자연어 처리를 하다보면 각 문장(또는 문서)은 서로 길이가 다를 수 있습니다. 그런데 기계는 길이가 전부 동일한 문서들에 대해서는 하나의 행렬로 보고, 한꺼번에 묶어서 처리할 수 있습니다. 다시 말해 병렬 연산을 위해서 여러 문장의 길이를 임의로 동일하게 맞춰 주는 작업이 필요할 때가 있습니다. 실습을 통해 이해해봅시다.

1. Numpy로 패딩하기

```
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
```

정수 인코딩 챗터에서 수행했던 실습을 그대로 반복해보겠습니다. 아래와 같이 텍스트 데이터가 있습니다.

```
preprocessed_sentences = [['barber', 'person'], ['barber', 'good', 'person'], ['barber', 'huge', 'p
erson'], ['knew', 'secret'], ['secret', 'kept', 'huge', 'secret'], ['huge', 'secret'], ['barber', '
kept', 'word'], ['barber', 'kept', 'word'], ['barber', 'kept', 'secret'], ['keeping', 'keeping', 'h
uge', 'secret', 'driving', 'barber', 'crazy'], ['barber', 'went', 'huge', 'mountain']]
```

단어 집합을 만들고, 정수 인코딩을 수행합니다.

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(preprocessed_sentences)
encoded = tokenizer.texts_to_sequences(preprocessed_sentences)
print(encoded)
```

모든 단어가 고유한 정수로 변환되었습니다.

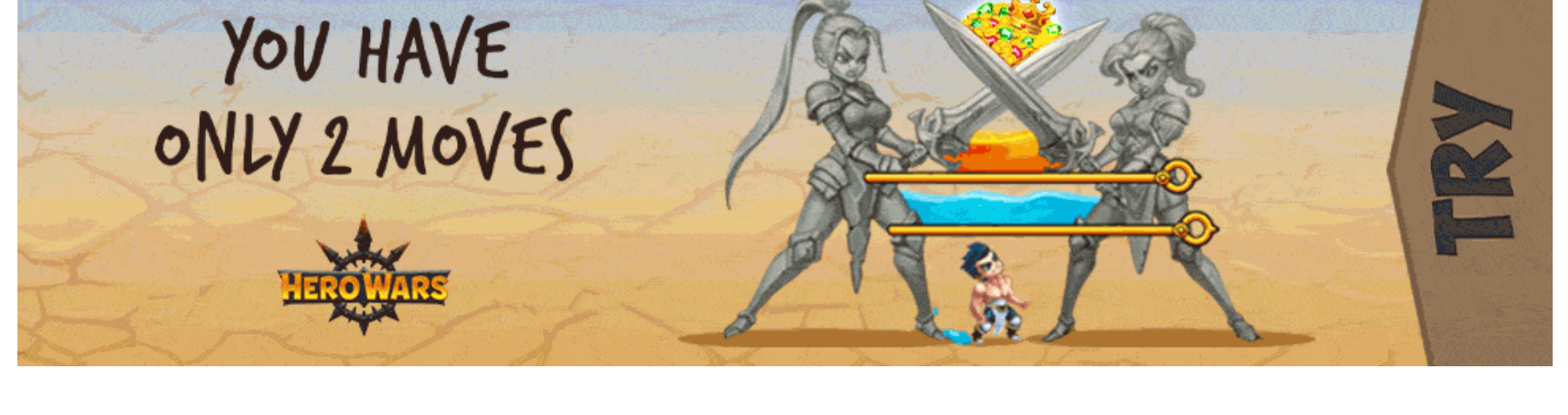
```
[[1, 5], [1, 8, 5], [1, 3, 5], [9, 2], [2, 4, 3, 2], [3, 2], [1, 4, 6], [1, 4, 6], [1, 4, 2], [7,
7, 3, 2, 10, 1, 11], [1, 12, 3, 13]]
```

모두 동일한 길이로 맞춰주기 위해서 이 중에서 가장 길이가 긴 문장의 길이를 계산해보겠습니다.

```
max_len = max(len(item) for item in encoded)
print('최대 길이 :', max_len)
```

```
최대 길이 : 7
```

가장 길이가 긴 문장의 길이는 7입니다. 모든 문장의 길이를 7로 맞춰주겠습니다. 이때 가상의 단어 'PAD'를 사용합니다. 'PAD'라 는 단어가 있다고 가정하고, 이 단어는 0번 단어라고 정의합니다. 길이가 7보다 짧은 문장에는 숫자 0을 채워서 길이 7로 맞춰줍니 다.



```
for sentence in encoded:
    while len(sentence) < max_len:
        sentence.append(0)
```

```
padded_np = np.array(encoded)
padded_np
```

```
array([[ 1,  5,  0,  0,  0,  0,  0],
       [ 1,  8,  5,  0,  0,  0,  0],
       [ 1,  3,  5,  0,  0,  0,  0],
       [ 9,  2,  0,  0,  0,  0,  0],
       [ 2,  4,  3,  2,  0,  0,  0],
       [ 3,  2,  0,  0,  0,  0,  0],
       [ 1,  4,  6,  0,  0,  0,  0],
       [ 1,  4,  6,  0,  0,  0,  0],
       [ 1,  4,  2,  0,  0,  0,  0],
       [ 7,  7,  3,  2, 10,  1, 11],
       [ 1, 12,  3, 13,  0,  0,  0]])
```

길이가 7보다 짧은 문장에는 전부 숫자 0이 뒤로 붙어서 모든 문장의 길이가 전부 7이된 것을 알 수 있습니다. 기계는 이들을 하나의 행렬로 보고, 병렬 처리를 할 수 있습니다. 또한, 0번 단어는 사실 아무런 의미도 없는 단어이기 때문에 자연어 처리하는 과정에서 기 계는 0번 단어를 무시하게 될 것입니다. 이와 같이 데이터에 특정 값을 채워서 데이터의 크기(shape)를 조정하는 것을 패딩 (padding)이라고 합니다. 숫자 0을 사용하고 있다면 **제로 패딩(zero padding)**이라고 합니다.

2. 케라스 전처리 도구로 패딩하기

케라스에서는 위와 같은 패딩을 위해 pad_sequences()를 제공하고 있습니다.

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

encoded 값이 위에서 이미 패딩 후의 결과로 저장되었기 때문에 패딩 이전의 값으로 다시 되돌리겠습니다.

```
encoded = tokenizer.texts_to_sequences(preprocessed_sentences)
print(encoded)
```

```
[[1, 5], [1, 8, 5], [1, 3, 5], [9, 2], [2, 4, 3, 2], [3, 2], [1, 4, 6], [1, 4, 6], [1, 4, 2], [7,
7, 3, 2, 10, 1, 11], [1, 12, 3, 13]]
```

케라스의 pad_sequences를 사용하여 패딩을 해봅시다.

```
padded = pad_sequences(encoded)
padded
```

```
array([[ 0,  0,  0,  0,  0,  1,  5],
       [ 0,  0,  0,  0,  1,  8,  5],
       [ 0,  0,  0,  0,  1,  3,  5],
       [ 0,  0,  0,  0,  0,  9,  2],
       [ 0,  0,  0,  2,  4,  3,  2],
       [ 0,  0,  0,  0,  0,  3,  2],
       [ 0,  0,  0,  0,  1,  4,  6],
       [ 0,  0,  0,  0,  1,  4,  6],
       [ 0,  0,  0,  0,  1,  4,  2],
       [ 7,  7,  3,  2, 10,  1, 11],
       [ 0,  0,  0,  1, 12,  3, 13]], dtype=int32)
```

Numpy로 패딩을 진행하였을 때와는 패딩 결과가 다른데 그 이유는 pad_sequences는 기본적으로 문서의 뒤에 0을 채우는 것이 아니라 앞에 0으로 채우기 때문입니다. 뒤에 0을 채우고 싶다면 인자로 padding='post'를 주면됩니다.



```
padded = pad_sequences(encoded, padding='post')
padded
```

```
array([[ 1,  5,  0,  0,  0,  0,  0],
       [ 1,  8,  5,  0,  0,  0,  0],
       [ 1,  3,  5,  0,  0,  0,  0],
       [ 9,  2,  0,  0,  0,  0,  0],
       [ 2,  4,  3,  2,  0,  0,  0],
       [ 3,  2,  0,  0,  0,  0,  0],
       [ 1,  4,  6,  0,  0,  0,  0],
       [ 1,  4,  6,  0,  0,  0,  0],
       [ 1,  4,  2,  0,  0,  0,  0],
       [ 7,  7,  3,  2, 10,  1, 11],
       [ 1, 12,  3, 13,  0,  0,  0]], dtype=int32)
```

Numpy를 이용하여 패딩을 했을 때와 결과가 동일합니다. 실제로 결과가 동일한지 두 결과를 비교합니다.

```
(padded == padded_np).all()
```

```
True
```

True값이 리턴됩니다. 두 결과가 동일하다는 의미입니다. 지금까지는 가장 긴 길이를 가진 문서의 길이를 기준으로 패딩을 한다고 가정하였지만, 실제로는 꼭 가장 긴 문서의 길이를 기준으로 해야하는 것은 아닙니다. 가령, 모든 문서의 평균 길이가 20인데 문서 1 개의 길이가 5,000이라고 해서 굳이 모든 문서의 길이를 5,000으로 패딩할 필요는 없을 수 있습니다. 이와 같은 경우에는 길이에 제 한을 두고 패딩할 수 있습니다. maxlen의 인자로 정수를 주면, 해당 정수로 모든 문서의 길이를 동일하게 합니다.

```
padded = pad_sequences(encoded, padding='post', maxlen=5)
padded
```

```
array([[ 1,  5,  0,  0,  0],
       [ 1,  8,  5,  0,  0],
       [ 1,  3,  5,  0,  0],
       [ 9,  2,  0,  0,  0],
       [ 2,  4,  3,  2,  0],
       [ 3,  2,  0,  0,  0],
       [ 1,  4,  6,  0,  0],
       [ 1,  4,  6,  0,  0],
       [ 1,  4,  2,  0,  0],
       [ 7,  7,  3,  2, 10],
       [ 1, 12,  3, 13,  0]], dtype=int32)
```

길이가 5보다 짧은 문서들은 0으로 패딩되고, 기존에 5보다 길었다면 데이터가 손실됩니다. 가령, 뒤에서 두번째 문장은 본래 [7, 7, 3, 2, 10, 1, 11]였으나 현재는 [3, 2, 10, 1, 11]로 변경된 것을 볼 수 있습니다. 만약, 데이터가 손실될 경우에 앞의 단어가 아니 라 뒤의 단어가 삭제되도록 하고싶다면 truncating이라는 인자를 사용합니다. truncating='post'를 사용할 경우 뒤의 단어가 삭 제됩니다.

```
padded = pad_sequences(encoded, padding='post', truncating='post', maxlen=5)
padded
```

```
array([[ 1,  5,  0,  0,  0],
       [ 1,  8,  5,  0,  0],
       [ 1,  3,  5,  0,  0],
       [ 9,  2,  0,  0,  0],
       [ 2,  4,  3,  2,  0],
       [ 3,  2,  0,  0,  0],
       [ 1,  4,  6,  0,  0],
       [ 1,  4,  6,  0,  0],
       [ 1,  4,  2,  0,  0],
       [ 7,  7,  3,  2, 10],
       [ 1, 12,  3, 13,  0]], dtype=int32)
```

숫자 0으로 패딩하는 것은 널리 퍼진 관례이긴 하지만, 반드시 지켜야하는 규칙은 아닙니다. 만약, 숫자 0이 아니라 다른 숫자를 패 딴을 위한 숫자로 사용하고 싶다면 이 또한 가능합니다. 현재 사용된 정수들과 겹치지 않도록, 단어 집합의 크기+1을 한 숫자로 사용해보시다.



```
last_value = len(tokenizer.word_index) + 1 # 단어 집합의 크기보다 1 큰 숫자를 사용
print(last_value)
```

```
14
```

현재 단어가 총 13개이고, 1번부터 13번까지 정수가 사용되었으므로 단어 집합의 크기+1을 하면 마지막 숫자인 13보다 1이 큰 14를 얻습니다. pad_sequences의 인자로 value를 사용하면 0이 아닌 다른 숫자로 패딩이 가능합니다.

```
padded = pad_sequences(encoded, padding='post', value=last_value)
padded
```

```
array([[ 1,  5, 14, 14, 14, 14, 14],
       [ 1,  8,  5, 14, 14, 14, 14],
       [ 1,  3,  5, 14, 14, 14, 14],
       [ 9,  2, 14, 14, 14, 14, 14],
       [ 2,  4,  3,  2, 14, 14, 14],
       [ 3,  2, 14, 14, 14, 14, 14],
       [ 1,  4,  6, 14, 14, 14, 14],
       [ 1,  4,  6, 14, 14, 14, 14],
       [ 1,  4,  2, 14, 14, 14, 14],
       [ 7,  7,  3,  2, 10,  1, 11],
       [ 1, 12,  3, 13, 14, 14, 14]], dtype=int32)
```

마지막 편집일시 : 2022년 11월 14일 2:43 오후

원하는 순위 노출정확도 98.3%

네이버 자동입찰 솔루션

목표순위와 최대입찰가만 설정하면 목표달성을 위한 가장 낮은 CPC를 자동으로 찾아줍니다.

주식회사 보라웨어

열기 >

원하는 순위 노출정확도 98.3%

네이버 자동입찰 솔루션

목표순위와 최대입찰가만 설정하면 목표달성을 위한 가장 낮은 CPC를 자동으로 찾아줍니다.

주식회사 보라웨어

열기 >

댓글 6 | 피드백

이전글 : 02-06 정수 인코딩(Integer Encoding)

다음글 : 02-08 원-핫 인코딩(One-Hot Encoding)

🔍

↑